



Anteater Poker Software Specification

Version 1.0 Alpha Draft

By Team 23

University of California, Irvine

EECS 22L

Note: This software specification document describes the system architecture, program structure, networking design, graphical interface design, communication methods, and major software components of the Anteater Poker application. The document outlines the planned implementation and behavior of the software system developed for EECS 22L. Anteater Poker is intended strictly for educational and entertainment purposes and does not involve real-money gambling.

Table of Contents

Glossary.....	3
1. Poker Software Architecture Overview.....	5
1.1. Main data types and structures.....	5
1.2. Major software components.....	5
1.3. Module interfaces.....	6
1.4. Overall program control flow.....	6
1.5. Poker Bot.....	6
2. Poker Server Architecture Overview.....	7
2.1. Main Data Types and Structures.....	7
2.2. Major software components.....	7
2.3. Module Interfaces.....	7
2.3. Overall program control flow.....	7
3. Installation.....	8
3.1. System Requirements and Compatibility.....	8
3.2. Unpacking and Configuration.....	9
3.3. Building, Compilation, and Installation.....	10
4. Documentation of Packages, Modules, and Interfaces.....	11
4.1. Detailed description of data structures.....	11
4.2. Detailed description of functions and parameters.....	12
4.3. Detailed description of input and output formats.....	12
5. Development Plan and Timeline.....	12
5.1. Partitioning of Tasks.....	12
5.2. Team Member Responsibilities.....	12
5.3. Weekly Timeline.....	15
Index.....	16
6. Back Matter.....	12
Copyright.....	12
Error messages.....	12
References.....	15
Index.....	16

Glossary

- GUI - Graphical User Interface
- TCP/IP - Communication protocol used for networking
- Socket - Communication endpoint between programs
- Client - Program used by a player to connect to the server
- Server - Central application that hosts the game
- Bot - AI-controlled poker player
- Community Cards - Shared cards used by all players
- Dealer - Player responsible for starting each round
- Anteater Wild Card - A special card that may represent any rank or suit
- Fold - To leave the current betting round
- Raise - To increase the current bet
- Call - To match the current bet
- Check - To pass a turn without betting

1. Poker Software Architecture Overview

1.1. Main Data Types and Structures

The Anteater Poker Client application uses various data structures to manage the gameplay, GUI updates, and player data. Arrays are used to store the collections of cards, community cards, player information, and active seats at the poker table. For example, a player's hand may be stored in a fix sized array containing two cards, while another array stores the five community cards shown during the round. Character Arrays are also used to store usernames, chat messages, and server addresses displayed in the GUI.

The game application also utilizes structures (struct) to organize related variables into single data types. A card structure may contain the ccard rank, suit, whether or not they have attained the special Anteater Wild Card, while a player structure may contain the player username, current points, seat number, and the cards in the player's hand. Pointers are also used through the client application to efficiently access shared data and the GTK components while the game is running. In addition, socket related data structures will be used to manage the TCP/IP communication between the client and server. One of the main structures used is the card structure.

1.2. Major Software Components

The Anteater poker client application is divided into various software components, where each component is responsible for a specific task within the poker application.

The main client module initializes the application, establishes the connection to the poker server, and manages the primary gameplay loop. The GTK GUI module manages all graphical interface elements such as the login screen, the poker table, betting buttons, community cards, and player information. The user input module processes player actions like calling, folding, raising, checking, and seat selection in the game.

Another major component is the display module, which updates information shown to the player during gameplay. This includes updating the community cards, pot values, player points, and current turns. The network communication module manages all TCP/IP socket communication between the client and server, including sending player actions, receiving game state updates, synchronizing gameplay information, and handling disconnections or communication errors.

1.3. Module Interfaces

The software modules communicate with one another through defined interfaces and function calls. The interfaces allow different components of the application to exchange data without directly accessing the internal module logic. The module

interface design improves overall performance and allows components to be modified independently.

The network communication module provides functions for connecting to the server, sending player actions, and receiving game updates. The GUI module interacts with the display module to update the GUI whenever new information is received from the server. The client application uses shared data structures to synchronize game information between the modules.

1.4. Overall Program Control Flow

The client application follows a continuous control flow throughout gameplay.

After the program launches, the GTK graphical interface is initialized and the player is prompted to enter connection information such as username or a server code. Once connected to the server, the client waits for updated game state information and displays the poker table interface to the player.

During gameplay, the client continuously receives updates from the server, refreshes the graphical interface, and processes user input. When the player performs an action such as calling, raising, folding, or checking, the client sends the action to the server and waits for updated game information. This process continues until the player disconnects or the server terminates the game session.

1.5. Poker Bot

The Anteater Poker client application includes support for automated Poker Bot players that can fill empty seats at the table. The Poker Bot is an AI-controlled

player capable of participating in gameplay using basic poker decision-making strategies. The bot can perform actions such as calling, raising, folding, and checking based on its current hand and the state of the game.

The Poker Bot is designed to simulate human gameplay behavior while maintaining simple and efficient logic suitable for the project. Bot players help maintain active gameplay even when fewer human players are connected to the poker table. The bot module interacts with the server similarly to a normal client and follows the same gameplay rules as human players.

2. Poker Server Software Architecture Overview

2.1. Main Data Types and Structures:

The poker server uses several major data types and structures to store and control game information. A Card structure stores information about each card, such as its rank, suit, and whether it is an Anteater Wild Card. A Deck structure stores all cards used in the game and keeps track of which cards have already been dealt.

A Player structure stores each connected player's username, socket connection, current points, current bet, seat number, hand cards, and status in the round. The player status may include active, folded, all-in, or disconnected. A Table structure stores the players seated at one table, the pot value, community cards, dealer position, current turn, and current betting round.

The server also uses arrays to manage multiple players, tables, cards, and sockets. Character arrays are used to store usernames, server messages, and commands received from clients. Pointers may be used to pass structures between server functions efficiently. These structures are similar to the client-side structures already described in the software architecture overview, but the server version focuses more on controlling the full game state and validating gameplay actions.

2.2. Major Software Components:

The server is divided into several major components. The first component is the connection manager, which creates the server socket, listens for incoming client connections, and assigns each player to a table or waiting area.

The second component is the game manager, which controls the poker round. It shuffles the deck, deals player cards, reveals community cards, tracks the betting rounds, updates the pot, and determines when the round is over.

The third component is the player action handler, which receives commands from clients such as fold, call, check, raise, or leave table. This component checks whether the action is legal before applying it to the game state.

The fourth component is the message handler, which sends updates from the server to the clients. These updates include player turns, card information, pot size, table status, chat messages, and round results.

The fifth component is the winner evaluation system, which compares player hands at the end of a round and determines the winner based on poker hand rules. If an Anteater Wild Card is used, the server must also apply the special card rule before deciding the final hand ranking.

2.3. **Module Interfaces:**

The server modules communicate with each other through clearly defined function calls and shared data structures. The connection module sends new player information to the table manager when a client joins the server. The table manager then updates the player list and assigns the player to an available seat.

The client and server communicate through text-based or structured messages sent over TCP/IP sockets. For example, a client may send a command such as CALL, RAISE, FOLD, or CHECK. The server reads the command, verifies it through the player action handler, updates the table data, and sends the result back to all affected clients.

The game manager communicates with the deck module to shuffle and deal cards. It communicates with the winner evaluation module to determine the winning player at the end of the round. The message handler communicates with all other modules because it is responsible for sending updated game information back to the connected clients.

Example server module interfaces may include:

```
int create_server_socket(int port);
```

```
int accept_player_connection(int server_socket);
```

```
void add_player_to_table(Player *player, Table *table);
```

```
void start_new_round(Table *table);
```

```
void process_player_action(Table *table, Player *player, char *command);
```

```
void broadcast_table_update(Table *table);
```

```
Player *evaluate_winner(Table *table);
```

```
void remove_disconnected_player(Table *table, Player *player);
```

2.4. Overall Program Control Flow:

The server program begins by loading the required server settings, such as the port number and maximum number of players. It then creates a server socket and begins listening for client connections. When a player connects, the server receives the player's username and assigns the player to a table.

Once enough players are seated, the server begins a new poker round. The deck is shuffled, each player receives their private cards, and the betting round begins. During each turn, the server waits for the current player to send an action. The

server checks the action, updates the game state, and broadcasts the new table status to all clients.

After the betting rounds are complete, the server reveals the remaining community cards and evaluates the hands of all active players. The winner receives the pot, and the server sends the final result to every client at the table. The server then resets the round data and prepares for the next hand.

If a player disconnects, the server updates the table and removes or marks the player as inactive. The server continues running until it is manually stopped. This control flow supports the document's goal of allowing players to connect to a valid game server and play Anteater Poker online using points only, not real money.

3. Installation

3.1. System Requirements and Compatibility

Anteater Poker is designed to run on a standard desktop or laptop computer and requires an operating system that supports C programming tools.

System requirements:

- Desktop or laptop computer

- Windows, macOS, or Linux operating system
- Stable internet connection for online play
- Working screen/display
- At least 512 MB of RAM recommended
- Access to a valid game server
- Updated operating system recommended for best compatibility

3.2. Unpacking and Configuration

The game can be installed through these options

Binary Package (Gameplay):

Extraction: % `gtar xvzf Poker_V1.0.tar.gz`

Open User Documentation: % `evince poker/doc/Poker_UserManual.pdf`

Start the server: % `poker/bin/server &`

Run the executable: % `anteaterpoker/bin/anteaterpoker`

Start the game:

1. Open the Anteater Poker client.
2. Enter a username or player profile.
3. Connect to an available Anteater Poker server.

4. Join or create a poker table.
5. Begin playing using points.

Source Code Package:

Extraction: % `gtar xvzf AnteaterPokerSource.tar.gz`

Open Software Documentation: % `evince anteaterpoker/doc/anteater_poker_software_specs.pdf`

Navigate to the project directory: % `cd anteaterpoker`

Compile with the Makefile: % `make`

Run the server: % `anteaterpoker/bin/server`

Run the client: % `anteaterpoker/bin/client`

Test with Makefile, if available: % `make test`

Clean compiled files, if needed: % `make clean`

Note:

For normal players, the binary package is recommended because it is already prepared for gameplay. The source code package is mainly used for testing, editing, or rebuilding the project. Make sure all commands are typed correctly and entered from the correct directory.

3.3. Building, Compilation, and Installation

After downloading and extracting the binary package, the player can open the client and connect to the server.

For source code installation, the program can be compiled using the Makefile: % make

After compilation, the executable files should be created in the program directory. The server should be started before the client so that players can connect successfully.

Common installation issues:

- Server is unavailable
- Internet connection is not active
- Incorrect server address or port number
- Missing game files
- Outdated version of the game
- Firewall or network settings blocking the connection

3.4 Uninstalling

To uninstall the program, please use the following Linux commands:

Go to the directory that contains the program: % cd directory_address_name

Delete the whole project directory: % rm -rf anteaterpoker

4. Documentation of Packages, Modules, and Interfaces

4.1. Detailed Description of Data Structures

The Anteater Poker application uses several important data structures to manage gameplay information, networking communication, player data, and graphical interface updates. Arrays are commonly used throughout the program to store collections of cards, player hands, community cards, player lists, and seat information. For example, a player's hand may be stored in a fixed-size array containing two cards, while another array stores the five community cards shown during gameplay.

The application also uses structures (structs) to organize related variables into single data types. A card structure stores information such as the card value, suit, and Anteater Wild Card status. A player structure stores information including the username, points, seat number, gameplay status, and hand of cards. Additional structures are used to manage socket communication, graphical interface components, game state information, and server-side poker tables.

EX:

```
1. typedef struct {  
  
    int value;  
  
    int suit;  
  
    int isWild;
```

```
} Card;
```

4.2. Detailed Description of Functions and Parameters

The Anteater Poker application uses multiple functions to manage gameplay, networking communication, graphical interface updates, and hand evaluation. Each module communicates through clearly defined function interfaces to improve software organization and maintainability.

The networking module contains functions responsible for establishing socket connections, sending player actions, and receiving game state updates from the server. The dealer module contains functions for shuffling the deck, distributing cards, and progressing betting rounds. The hand evaluation module contains functions that compare poker hands and determine the winning player during the showdown phase. The graphical interface module contains functions responsible for updating displayed cards, player points, betting controls, and status messages shown through GTK

Some Function Prototypes:

```
void shuffle_deck(Card deck[]);
```

```
void deal_cards(Player players[], Card deck[]);
```

```
void send_game_update(int socket_fd);
```


Each function receives parameters related to player information, socket communication, game state management, or graphical interface updates depending on the responsibility of the module.

4.3. Detailed Description of Communication Protocol

Anteater Poker uses TCP/IP socket communication to exchange information between the client and server applications. The server acts as the central controller of the game and continuously communicates with all connected clients to synchronize gameplay information. Communication occurs through socket-related function calls.

The communication protocol is responsible for transmitting player actions, gameplay updates, card distributions, betting information, and connection status messages between the server and connected clients. When a player performs an action such as calling, raising, folding, or checking, the client sends a message to the server. The server validates the action, updates the game state, and broadcasts the updated information to all connected clients.

The application may use predefined protocol flags or message identifiers to simplify communication between modules. Example communication messages include:

- * PLAYER_JOIN

- * PLAYER_ACTION

* ROUND_START

* DEAL_CARDS

*GAME_OVER

5. Development Plan and Timeline

5.1. Partitioning of Tasks

The project tasks are divided into several major categories to improve team organization and development efficiency. These categories include networking communication, graphical user interface development, poker gameplay logic, hand evaluation, bot development, testing, debugging, and documentation.

5.2. Team Member Responsibilities

Each team member is responsible for every portion of the Anteater Poker project.

We all contribute to it at the same time as delegation of different modules will come with time.

5.3. Weekly Timeline

The Anteater Poker project follows a development timeline that corresponds with the EECS 22L project deliverables. During the early weeks of development, the team focuses on software planning, software architecture, and documentation.

Middle development phases focus on gameplay implementation, networking communication, graphical interfaces, and Poker Bot integration. Final

development phases focus on testing, debugging, optimization, and final release preparation.

6. Back Matter

Copyright:

Anteater Poker was developed for educational purposes as part of EECS 22L at the University of California, Irvine. All source code, documentation, graphical assets, and related project materials are property of Team 23 unless otherwise stated. The project is intended strictly for academic and educational use.

References:

The Anteater Poker project references concepts from Texas Hold'em poker rules, TCP/IP networking documentation, GTK graphical user interface libraries, and EECS 22L course materials. Additional technical references may include Linux socket programming documentation, GCC compiler documentation, and software engineering design principles discussed throughout the course.

Index:

Anteater Wild Card — Special wildcard card used in Anteater Poker gameplay

Bot Player — AI-controlled poker player

Client — Player-side poker application

Community Cards — Shared poker cards visible to all players

Dealer Module — Server module responsible for card distribution

GUI — Graphical User Interface

Networking Module — Module responsible for socket communication

Player Structure — Data structure storing player information

Poker Bot — Automated player logic

Server — Central application managing gameplay

Socket Communication — TCP/IP communication between client and server

Texas Hold'em — Base poker rules used in Anteater Poker